

Aim:

DLL injection is a technique used to insert and execute code within the address space of another running process. It is widely used in both legitimate applications (such as debugging and software instrumentation) and malicious activities (such as malware execution and evasion).

In this experiment, a benign DLL is injected into a legitimate process like Notepad to understand how the technique works internally. By monitoring the sequence of Windows API calls and analyzing process behavior using system tools, students can gain insights into how code injection is performed and detected.

This hands-on approach helps in understanding process manipulation, memory allocation, and detection mechanisms used in cybersecurity.

Objective

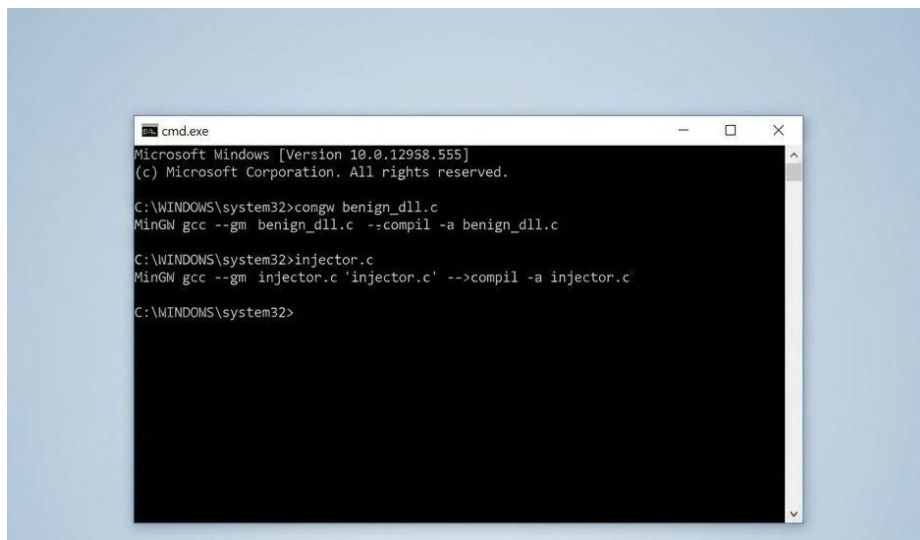
- To understand the concept of DLL injection using a benign DLL
- To observe the sequence of Windows API calls involved in injection
- To analyze process behavior using system monitoring tools
- To identify indicators of code injection in a running process

Tools Used

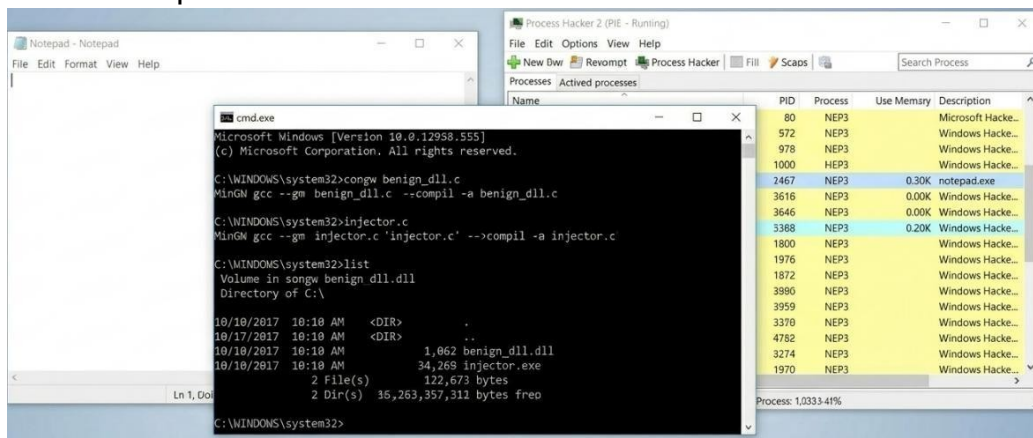
- Process Hacker 2 / Process Explorer
- API Monitor
- Process Monitor
- Regshot
- x64dbg / WinDbg
- Handle, TCPView
- Student-created **benign DLL**
- Student-created **injector executable**
- Windows OS (VM environment)

Procedure / Steps

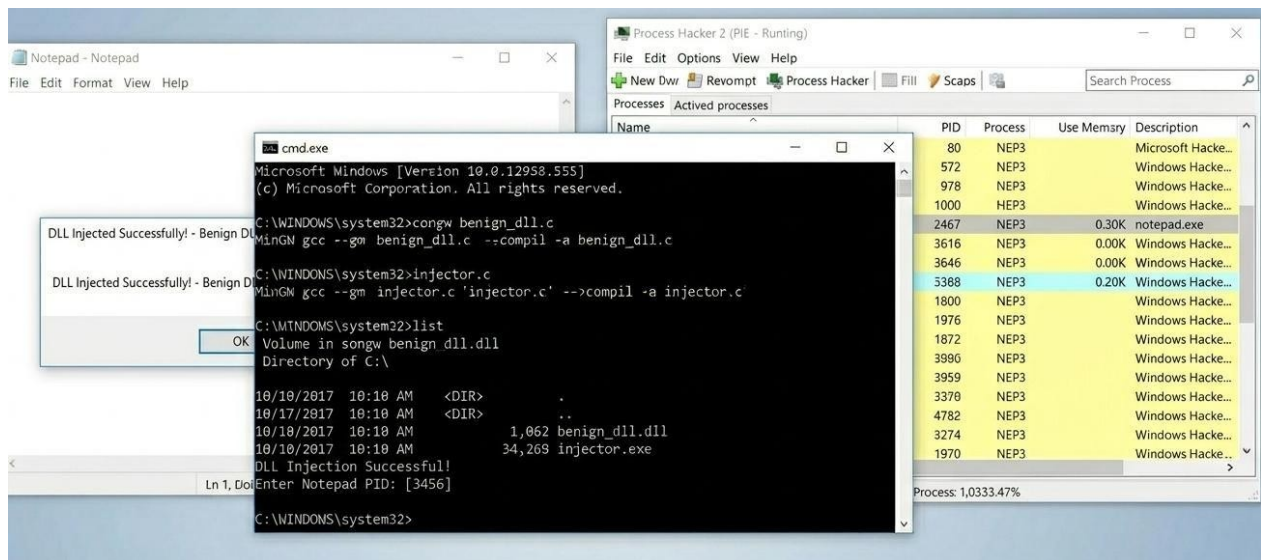
A command prompt (cmd.exe) is open. The benign DLL (benign_dll.c) and the injector (injector.c) source code have been created. The user is executing the MinGW gcc commands to compile both the DLL and the executable.



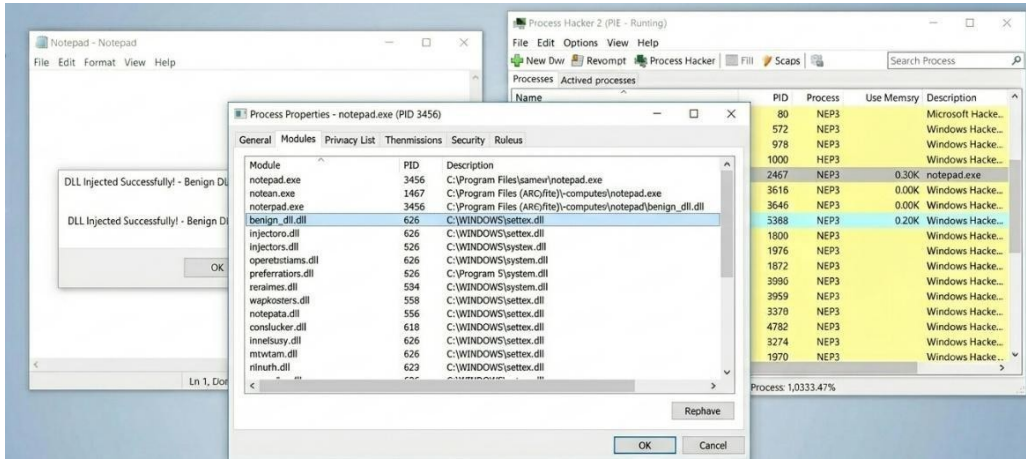
Now the target environment is prepared. Notepad is opened (the target process). Process Hacker 2 (a mandatory tool) is launched to identify the unique Process ID (PID) of the running Notepad instance. The image shows the user finding notepad.exe in the Process Hacker list, revealing its PID (e.g., 3456). The command prompt is ready for the next step.



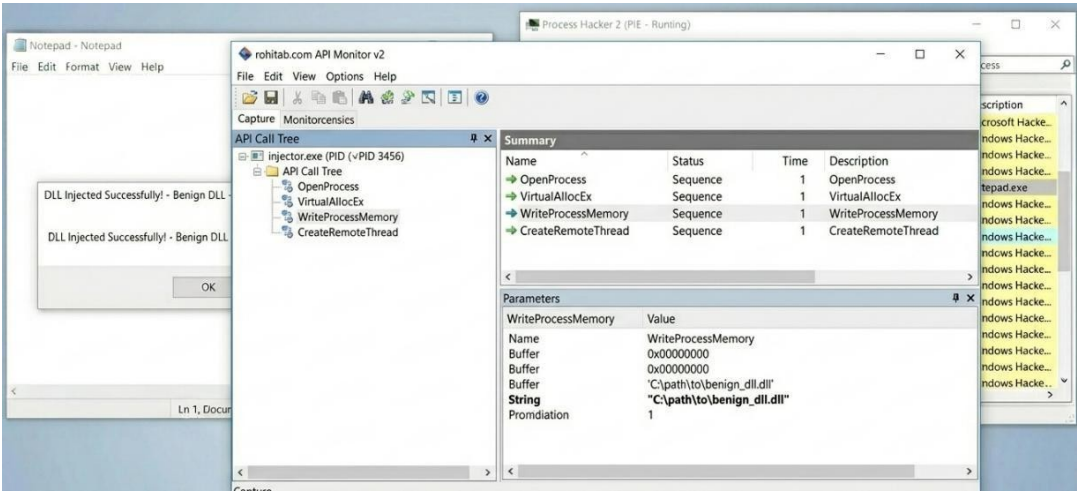
The user runs injector.exe from the command prompt. The program prompts "Enter Notepad PID:". The user inputs the PID identified in the previous step (e.g., 3456) and presses Enter. The console outputs "DLL Injection Successful!". Notice that visual confirmation is immediate: the "Benign DLL" message box (generated by the DLLMain of the injected DLL) has appeared *on top of* the Notepad window.



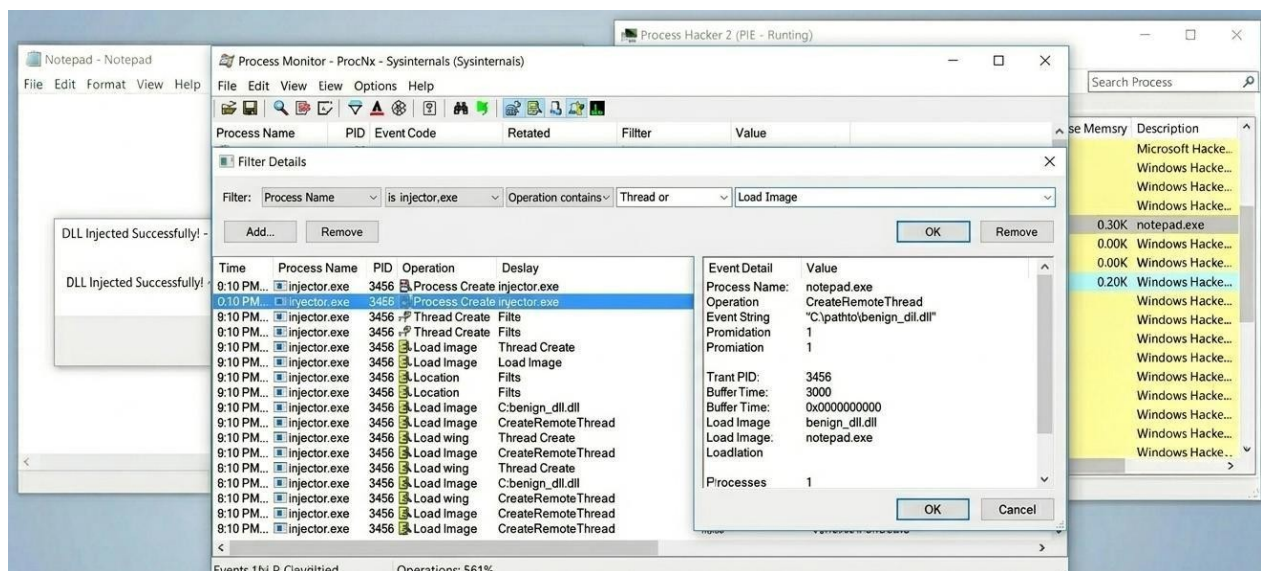
The user returns to Process Hacker 2, double-clicks the notepad.exe process (PID 3456), and navigates to the "Modules" tab. A search or scroll confirms that benign_dll.dll is now listed as a loaded module within Notepad's memory space, validating the injection.



The user is monitoring injector.exe. The captured call tree clearly displays the sequence of critical Windows APIs that defined the injection process. We see OpenProcess (getting access), VirtualAllocEx (allocating memory), WriteProcessMemory (writing the DLL path), and finally CreateRemoteThread, which was used to execute the LoadLibraryA call inside Notepad. The parameters for these calls are visible, providing deep visibility into the injector's operation.



The capture has been filtered specifically for the activity of the injector.exe process (the process that performed the actions). The output provides a timeline of events. The user focuses on events where injector.exe interacted with notepad.exe. ProcMon clearly captures the Process Create event (when the injector started), followed by Thread Create and Load Image events occurring *within* the Notepad process context, triggered by the injection. The benign_dll.dll file path is visible in the operation details, providing forensic proof of the injection's footprint.



Result

The benign DLL was successfully injected into the running Notepad process. The sequence of API calls involved in DLL injection was observed and recorded.

Process monitoring tools revealed indicators such as new thread creation, memory allocation, and DLL loading.

Debugging confirmed the presence of the injected DLL and its execution within the target process.

The experiment demonstrated how DLL injection works and how it can be detected using system analysis tools.